City Library After-School Program

LIBRARY REPORT

PREPARED BY :
RAGHAD MOHAMED

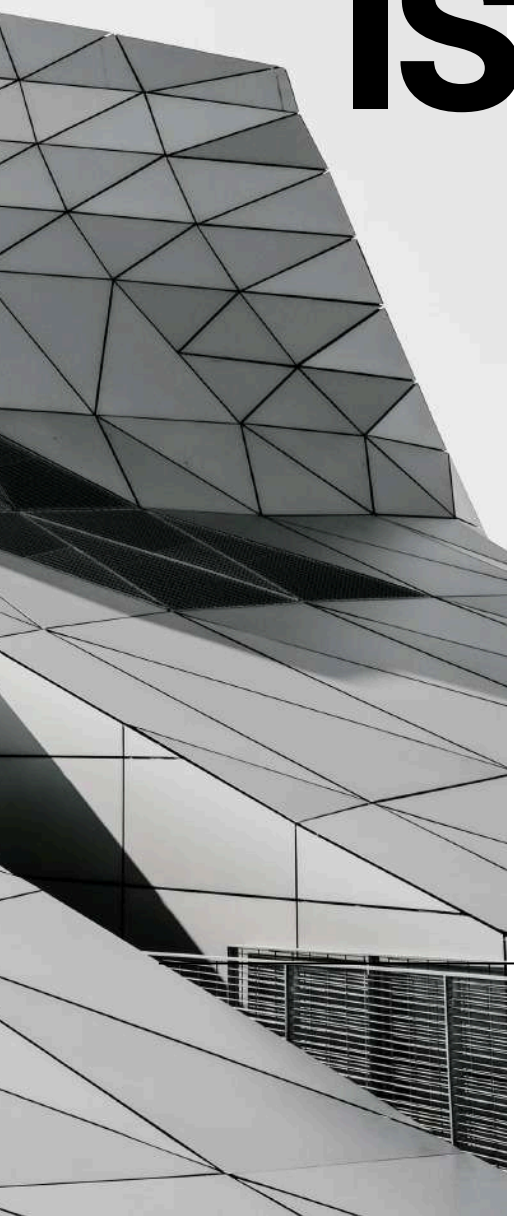
DECI EYOUTH LEVEL 3 Data Science and AI Final Project

TABLE OF CONTENT

ABOUT TASK 2

Task 2 is called Data Integrity, and it operates entirely on **task1_combined_data.csv** — the file produced by Task 1. Four separate quality problems are hiding somewhere in that combined dataset. My job is to find each one, decide how to handle it, and be ready to explain why you made that choice. Once that's done, two deliverables result from this task: a cleaned dataset and a written report describing what you found.

ABOUT DATA QUALITY ISSUES



Before any dataset can be judged, compared, or trusted, it needs to be reviewed carefully for quality and consistency. A dataset that looks fine at a glance can still be hiding gaps, repeats, and inconsistencies that would quietly distort anything built on top of it later.
That's exactly what this task asks of you: review the data carefully, on its own terms, before Task 3 tries to draw any conclusions from it.

DATA INSPECTION

I examined the dataset's 1st 5 rows and the last 5 rows.

Implementation: df_clean.head(n=5) and df_clean.tail(n=5)

I examined the shape & the size of the df.

Implementation: df_clean.size and df_clean.shape

I used df_clean.info() to get general information about the df

I used df_clean.describe() to get a statistical description of the df

I listed the Columns' names in a list

Implementation: df_clean.columns.to_list()

Calculated the number of duplicated rows & got them

Implementation: df_clean.duplicated().sum()
and df_clean[df_clean.duplicated()]

Calculated the number of missing values in each column

Implementation: df.isnull().sum()

I got the data type of each column

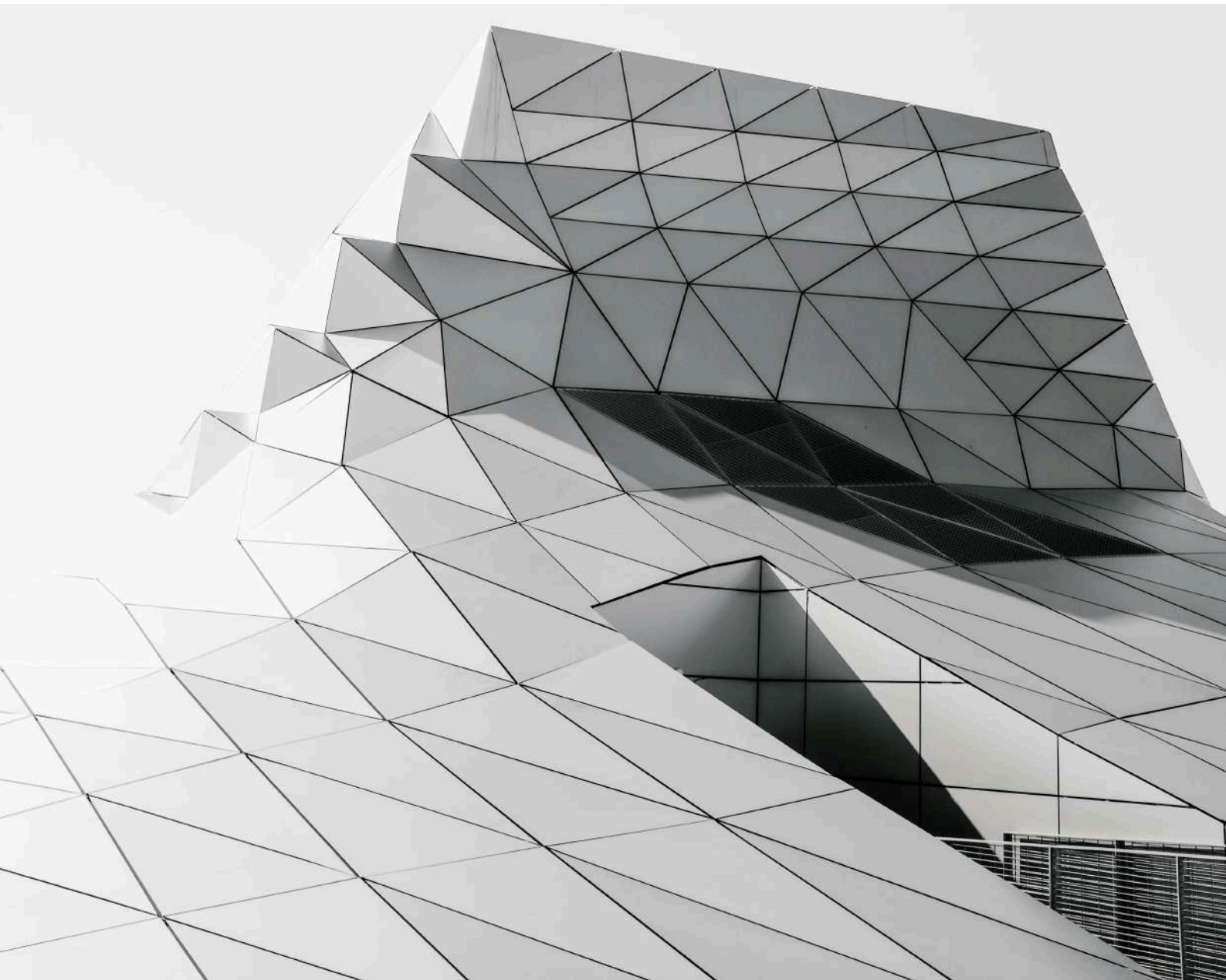
Implementation: df_clean.dtypes

AND AFTER THE INSPECTION PROCESS I FOUND 3 PROBLEMS.....

MISSING VALUES
DUPLICATED RECORDS
INCONSISTENT FORMATTING

City Library After-School Program

DATA CLEANING



DUPLICATED ROWS

Example on duplicated rows

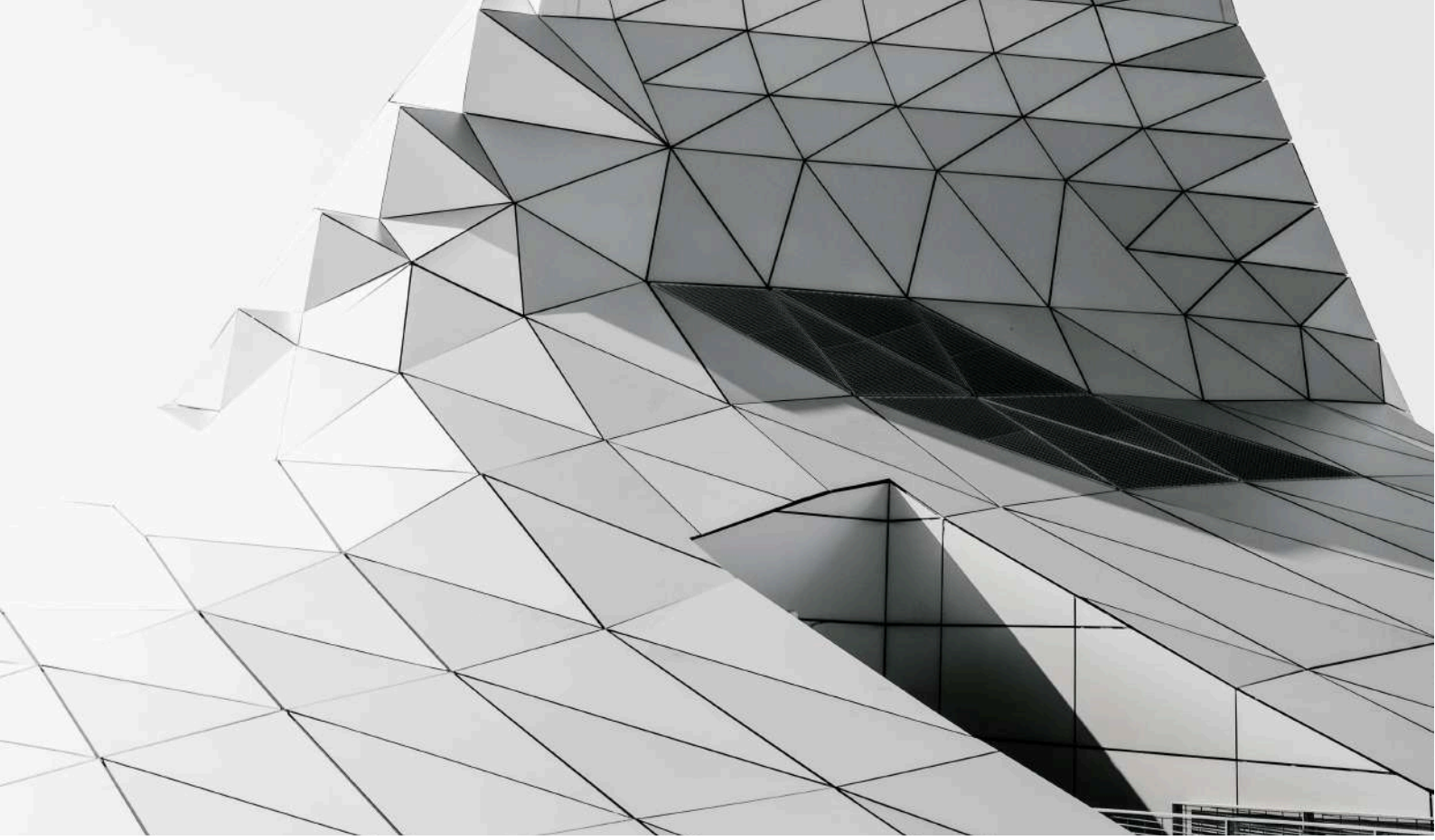
Name	ID	Email
Alice	101123	alice@gmail.com
Alice	101123	alice@gmail.com

The Basis of duplicated rows :

Fully duplicated rows : Must be removed because they may cause wrong information
Partially duplicated rows : These must be kept because they represent unique events. The same rule applies when we break down the data at the person level

And I got rid of the fully duplicated rows by :

```
df_clean.drop_duplicates(keep="first")
```



Output Asset :

A new cleaned dataset cleaned from duplicates

Verification :

Then I inspected the new dataset if it had any other duplicates and it was cleaned from the duplicates successfully. After getting rid of the duplicates let's get to the next step

INCONSISTENT FORMATTING

Execution Strategy : I began by correcting the formatting first to prevent any errors during filling the missing values in the dataset df_clean

The Root Issue : Type mismatch within single dataset columns

The Conflict : Raw data mixes pure integers, floating points, and text strings (e.g., 1011 vs "1011" vs 1011.0)

The Operational Impact : Pandas defaults these mixed columns to the generic object dtype, which completely breaks downstream imputation routines and mathematical aggregations

The Solution : Formatting must be completely unified across all fields before handling any missing values in df_clean

Type Casting Operations

Target Fields : member_id - checkout_id - book_id

Transformation Breakdown :

Step 1 (Numeric Cleanup): object/int/float → numeric
What it does : Takes messy text strings and decimals, forces them into raw numbers, and cleans up data type conflicts

Step 2 (Integer Unification): numeric → pd.Int64Dtype
What it does: Formats those numbers into a clean integer structure (Int64), so decimals disappear while keeping missing values intact

Step 3 (Final String Storage): pd.Int64Dtype → STR
What it does: Converts the clean integers into uniform text strings for long-term storage and changes missing value tags to np.nan

The code

```
# Phase 1 & 2: Convert to numeric, handle mixing, and shift to clean Integers
df_clean["col"] = pd.to_numeric(df_clean["col"], errors="coerce").astype(pd.Int64Dtype())
# Phase 3: Turn the Int to uniform str & the Nan str to np.nan
df_clean["col"] = df_clean["col"].astype(str).replace("<NA>", np.nan)
```

Type Casting Operations

Target Fields : first_name , last_name , neighborhood , membership_status , genre , Publisher

Transformation Breakdown :

Step 1 (Text Standardisation): object/mixed → str.lower().str.strip().str.title()
What it does : Converts text data to uniform strings, stripped out extra spaces, and applies Title Case styling to fix inconsistent capitalisations.

Step 2 (Null Mapping): "Nan" / "None" string → np.nan
What it does : Finds any literal text strings that say "Nan" or "None" and maps them directly into proper numpy missing value formats.

The code example

```
text_cols = ["first_name", "last_name", "neighborhood", "membership_status", "genre", "Publisher"]
for col in text_cols:
    # Step 1: Clean casing and strip extra padding spaces
    df_clean[col] = df_clean[col].astype(str).str.lower().str.strip().str.title()
    # Step 2: Map placeholder text artifacts back to true nulls
    df_clean[col] = df_clean[col].replace({"Nan": np.nan, "None": np.nan})
```

Type Casting Operations

Target Fields : grades - pages - publication_year - dates (join_date, return_date, checkout_date)

Transformation Breakdown :

- **Step 1 (grades):** Float → int → STR
  - **What it does :** Converts grades from floats to integers to prevent the trailing decimal point (.0) from showing up when stored as a final nominal string.
- **Step 2 (Pages):** Float → int
  - **What it does :** Drops decimal fractions from the layout to parse book page configurations straight into integer formats.
- **Step 3 (Publication year):** Float → int → STR
  - **What it does :** Casts publication years cleanly through integers into strings so they are ready for targeted filtering and math calculations.
- **Step 4 (dates):** object → pd.to_datetime
  - **What it does :** Enhances loose string dates by transforming them directly into high-utility native datetime series.

The code example

```
# 1. grades: Float to int to clean string (removes .0 decimal artifact)
df_clean["grades"] = pd.to_numeric(df_clean["grades"], errors="coerce").astype(pd.Int64Dtype()).astype(str).replace("<NA>", np.nan)

# 2. Pages: From Float format to true integer
df_clean["Pages"] = pd.to_numeric(df_clean["Pages"], errors="coerce").astype(pd.Int64Dtype())

# 3. Publication year: From float to int to clean string representation
df_clean["Publication year"] = pd.to_numeric(df_clean["Publication year"],
                                             errors="coerce").astype(pd.Int64Dtype()).astype(str).replace("<NA>", np.nan)

# 4. dates: Object text to native pandas datetime objects
date_cols = ["join_date", "return_date", "checkout_date"]
for col in date_cols:
    df_clean[col] = pd.to_datetime(df_clean[col], errors="coerce")
```

MISSING VALUES

Diagnostic Steps : I got the number of missing values in each column & the percentage of missing values in each column to decide what to do with the rows & the columns.

Imputation Matrices :

Mode Imputation : neighborhood, grade, membership_status, join_date, and checkout_date were filled with The mode
Mean Imputation : pages → mean
Median Imputation : publication_year → median
Static Placeholder (Unknown) : Publisher → Unknown
Static Placeholder (Returned Status) :
return_date → Not Returned
Static Placeholder (Availability) :
last_name , first_name → Not Available
Static Placeholder (Missing Status) :
book_id, checkout_id → Missing
Static Placeholder (Lost or Missing) :
member_id → Lost ID
Contextual Imputation : Genres → Filled based on the mode of genre to their corresponding book_id, & any remaining nans were filled with the overall mode of the genre column.

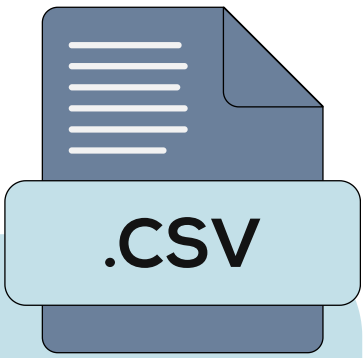
DATA CLEANING

Post-Imputation Audit :

Then, I re-inspected the dataset & there weren't any missing values or problems & the cleaning process was completed successfully. & there wasn't any problem in the cleaned dataset df_clean.

Deployment Path :

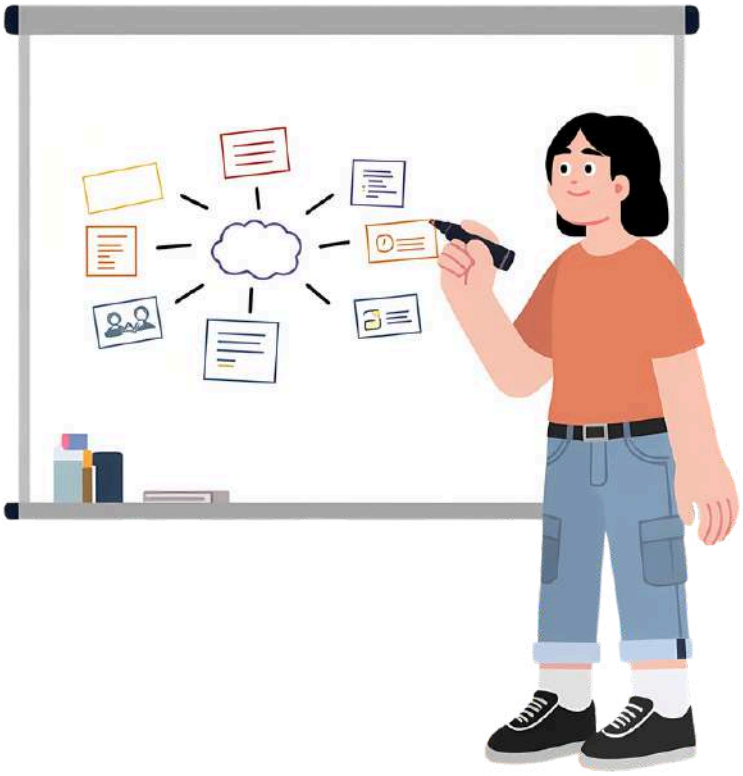
The cleaned dataset was saved in :



`311091_20100123_Library_task2_cleaned data.csv`

DATA MERGING

Finally, after the cleaning process, we went to the Visualisation Part. Before we went to the visualisation part, I had a point to make.
The problem of the missing members' IDs was solved already in task 1 during the merging process, so when I was merging, I used a new technique to solve this problem, so I will demonstrate what I did now.



Members and Checkouts Merging :

To merge the 2 tables in the database, I applied an outer join (how = “outer”) to capture all the records of the members table and the checkout table

SQL & JSON Integration :

To combine the datasets, I applied a left join (how="left") to merge the SQL tables with the JSON data, ensuring book descriptions were retrieved exclusively for books already present in the SQL tables.

HTML Table Aggregation :

Next, I merged this combined SQL-JSON dataset with the HTML table using an outer join (how="outer"), capturing all information from both sources to ensure maximum data retention.

Validation Matrix :

An inspection of the final dataset confirmed zero missing values for member_id, verifying that 100% of the IDs from both the SQL members table and the HTML table were successfully included.

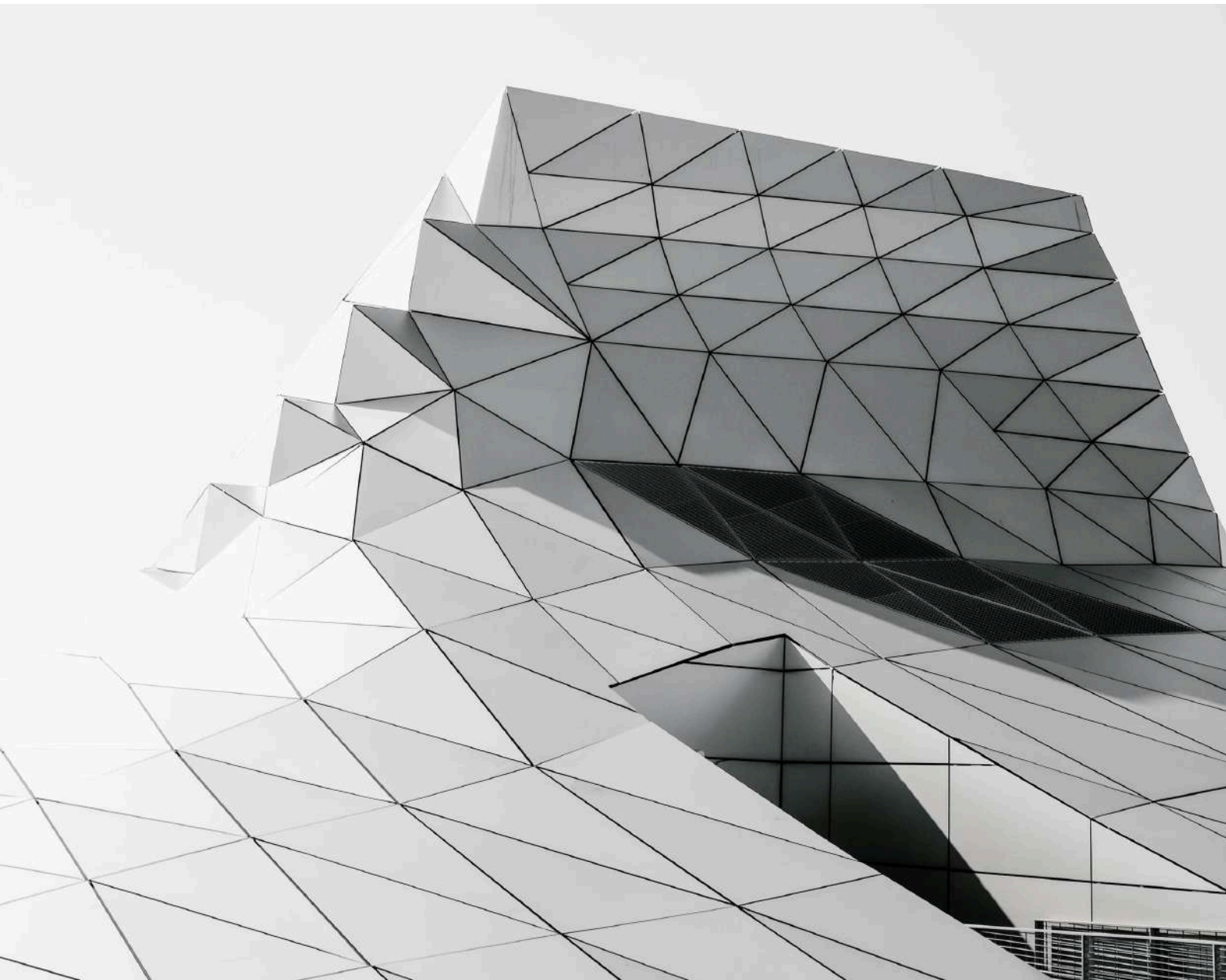
ANALYTICAL TARGET

I made 5 goals

g1 : Which Publishers have the most books checked out?
g2 : Which book genres had the highest circulation?
g3 : What is the distribution of the various memberships statuses amongst the borrowers?
g4 : What are the busiest periods for checkouts?
g5 : Which neighborhoods have the most active members (highest average checkouts per member)?

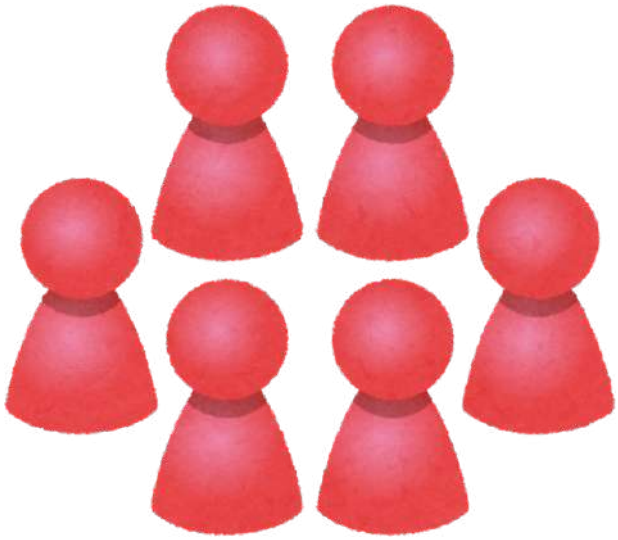
Business Value : This comprehensive process ensured the dataset was robust, consistent, & ready for further analytical exploration & provided meaningful insights into the library's operations.
Reporting Output : Then, I generated insights & put them in the Colab notebook.

SHORT SUMMARY



DUPLICATED ROWS

What I found :
Identical rows present in the dataset (8 identical rows)
Where it was : Across the entire DataFrame.
How big it was : 8 duplicated rows (row 101-row 128-
row 202-row 210- row 247-row 291-row 343-row 348)
What I did & why : I used
df_clean.drop_duplicates(keep="first") to remove these exact duplicates, keeping the first occurrence, as they represented redundant entries.



INCONSISTENT FORMATTING

What was found : Incorrect data types & inconsistent casting in various columns. ex: dates as objects, IDs as floats, etc, as we showed at the beginning.
Where it was : In columns such as member_id, checkout_id, book_id, first_name, last_name, grade, neighborhood, membership status, genre, publisher, join_date, checkout date, return_date, pages, publication year.
How big it was : Across the entire dataset except for total_books_checked_out .

IDs (member_id, checkout_id, book_id) :
To prevent floating-point artefacts (e.g., 101.1) from corrupting identifier values, these fields were first cast to nullable integers using pd.Int64Dtype(). This preserves NaN entries without forcing a decimal representation. The nullable integers were then converted to str, with missing values explicitly mapped to "np.nan" to standardise handling during the next filling step.

Text fields (first_name, last_name, neighborhood, genre, publishers, membership_status):
Consistency was enforced by casting to str, stripping whitespace, and applying uniform casing (e.g., title case for names). Multiple consecutive spaces were collapsed into single spaces in neighborhood and membership_status to eliminate formatting irregularities.

Numerical fields (grades, pages, publication_year) :
To retain integer precision while accommodating missing data, these fields were converted to pd.Int64Dtype(). grade was then converted to a string. Publication year was kept in integer format to preserve its utility for mathematical calculations across different time periods.

Date fields (join_date, checkout_date, return_date) :
Heterogeneous date formats were normalised by parsing with pd.to_datetime(format="mixed", errors="coerce"). This safely converts unparseable entries to NaT (Not a Time) without raising errors, ensuring a uniform datetime structure across the dataset.

MISSING VALUES

What was found: A significant number of NaNs across many columns.
Where it was: Notably in return_date (25.06%)

How big it was :

1. return_date — 25.53%
2. publication_year — 17.8%
3. grade — 14.99%
4. checkout_id, genre, pages, publisher — Around 10%
5. join_date — 7.49%
6. first_name, last_name, neighborhood, membership_status — Around 6%

```
The percentage of the missing values for each column :
 member_id                   0.0
first_name                  6.09
last_name                   6.09
grade                      14.99
neighborhood                6.09
membership_status           6.09
join_date                   7.49
checkout_id                 10.3
book_id                     4.22
checkout_date               4.22
return_date                25.53
genre                       10.3
pages                       10.3
publication_year            17.8
publisher                   10.3
total_books_checked_out      0.0
```

Range Summary :

Ranged from (4.14%) (book_id, checkout_date) to (25.06%) (return_date) of records in affected columns.

What was done and why :

neighborhood, grade, checkout_date, membership_status, join_date :

These fields were imputed using their respective modes. This approach is analytically sound because all five variables are either categorical or temporal; the most frequently occurring value provides a representative placeholder without distorting the underlying distribution.

pages :

Missing values were filled with the rounded mean. For numerical data without extreme outliers, the mean offers a stable central estimate that preserves the overall scale of the dataset.

publication_year :
The median was selected over the mean for imputation. The mean risks producing a non-integer (e.g., a float year), whereas the median yields a discrete, interpretable value that better reflects a typical publication year.

publisher & return_date :
Publisher was filled with "Unknown" and return_date with "Not Returned". These are informative, assumption-free placeholders that clearly signal missing information rather than fabricating data.

first_name & last_name :
Both were filled with "Not Available". This explicitly flags absent information from the source while ensuring the columns remain free of NaN for downstream analysis. At a later stage, students can be contacted using their member_id to collect the missing details, or an alternative database can be queried.

genre :

A two-step contextual imputation was applied. Missing values were first filled by the **mode of genre for the corresponding book_id**, leveraging existing relationships within the data. Any remaining NaNs were then filled with the **overall mode of the genre column** to maintain maximum genre consistency.

checkout_id & book_id :

These were filled with "Missing" to flag their absence explicitly. This preserves the columns as str for type consistency and prevents computational errors in subsequent operations.

MERGING PROCESS

“member_id”

Structural Alignment (Checkouts with no matching members) :
The members table and the checkouts table were merged using how="outer". This retains all the records in both tables without any deletion.

Dataset Unification :
The combined SQL tables were then merged with the JSON dataset using how="left". This ensures that book information is attached only where a valid member_id and book_id exist in the combined DataFrame.

HTML Structural Outer Join :
The combined SQL-JSON DataFrame was merged with the HTML table using how="outer". This preserves all records from both sources and connects matched member_ids while retaining unmatched HTML entries.

Post-Join State :
The resulting DataFrame contains HTML records with only three populated fields (member_id, book_id, checkout_id), with all other columns as NaN. Of the 22 rows originating from the HTML table, only 3 were not found in the members table.
What was found : 3 members were not present in the members table in the SQL database. (1104 , 1150 , 1201)
Where it was : across the entire table.
How big it was : there were 3 rows.
What I did and why : I decided to keep them to benefit from the rows in the analysis process and to use them if we needed to get the number of new members every month or year.
Strategic Retention :
The remaining NaNs were addressed during the missing-value imputation stage. These records were deliberately retained because they contribute to library analysis and can inform operational decisions aimed at improving library development.

This Report was an integrity report for task 2 that included what I did in details.

THANKS